ScyllaDB

TEAM SESSION · NOVEMBER 2026

Argus and SCT

From reading a test page in Argus to running, adjusting and writing SCT tests.

Part 1 · Argus, live Part 2 · Argus and Jenkins Part 3 · SCT

Three parts

PART 1 · LIVE

Argus in practice

Shown in the browser, no slides: a test page, one run, the Run button, and Clone.

PART 2 · UNDER THE HOOD

Argus and Jenkins

- Tracker vs executor
- A job is a small file
- How SCT reports back

PART 3 · SCT

Inside a test

- What one run does
- Class, YAML, pipeline
- Ways to run, backends
- Edit or write a test

Examples come from the performance tests we run, where they are general enough.

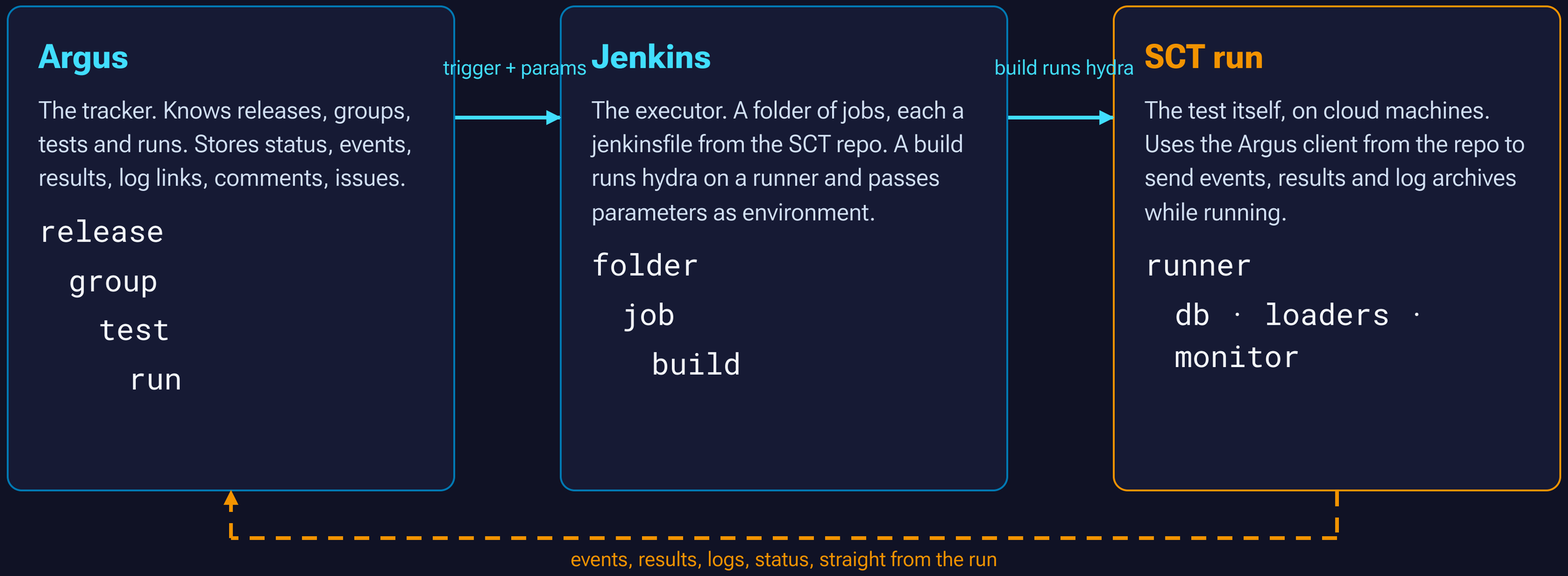
PART 1 · LIVE IN ARGUS

Switching to the browser

1. Read a test page
2. Read one run: events, results, logs
3. Run again with changed parameters
4. Get your own page by cloning

PART 2 · UNDER THE HOOD

Argus tracks. Jenkins executes. SCT reports.



A run in Argus and a build in Jenkins are the same thing seen from two sides. The Jenkins link on the run header takes you across.

PART 2 · UNDER THE HOOD

A Jenkins job is a small file in the SCT repo

JENKINS-PIPELINES/OSS/LONGEVITY/LONGEVITY-10GB-3H.JENKINSFILE

```
longevityPipeline(
  backend: 'aws',
  region: 'eu-west-1',
  test_name:
'longevity_test.LongevityTest.test_custom_t
  test_config: 'test-
cases/longevity/longevity-10gb-3h.yaml',
)
```

That is the whole job: which shared pipeline, which backend, which test class, which YAML. The Argus test is a page that points at this job.

Shared pipelines live in vars/ in the SCT repo. More in docs/sct-pipelines.md.

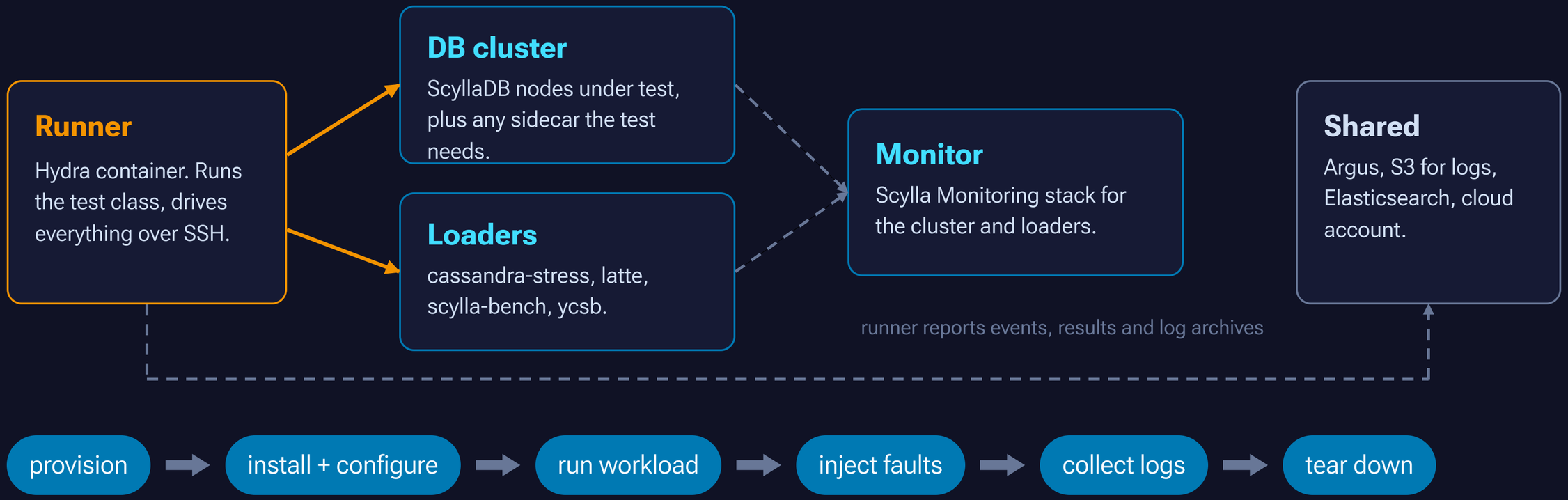
WHAT THE PIPELINE DOES ON EVERY BUILD

1. Create the run in Argus
2. Create an SCT runner VM in the cloud
3. Run hydra there with the job's settings plus yours
4. Collect logs, clean up resources
5. Report final status to Argus and by e-mail

Steps 4 and 5 run even when the test fails or is aborted.

PART 3 · SCT

One SCT run, end to end



The SCT README has a generated system map per test type (docs/system-map) that shows exactly this for performance, longevity, upgrade and others.

PART 3 · SCT

A test is three files and a config stack

Test class

`longevity_test.py`

Python. Inherits the cluster tester, decides what to run and what to report.

Test-case YAML

`test-cases/longevity/longevity-10gb-3h.yaml`

Node counts, sizing, stress command, duration, metadata. Most changes stop here.

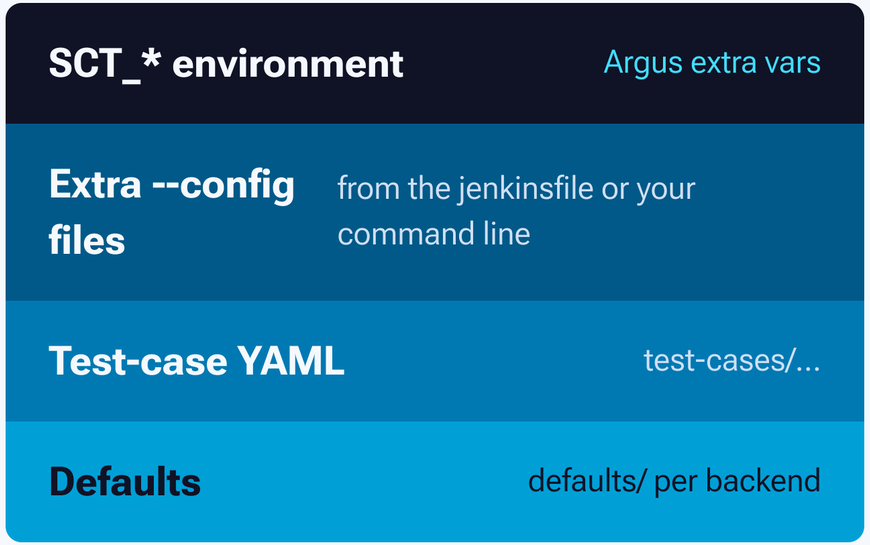
Jenkins pipeline

`jenkins-pipelines/oss/longevity/longevity-10gb-3h.jenkinsfile`

Binds the YAML to a backend, a region and a pipeline template.

Walkthrough for a new test: docs/writing-a-test.md.

CONFIG LAYERS, TOP WINS



The extra variables typed into Argus in Part 1 sit on top of this stack. Every option is documented in docs/configuration_options.md.

PART 3 · SCT

Three ways to run a test

IN THE CLOUD

Argus or Jenkins

Run button in Argus, or Build with parameters in Jenkins. A runner VM is created next to the cluster. Results tracked in Argus.

Where to look while it runs: the Jenkins console first, then the Argus run as events arrive.

Needs

- Argus and Jenkins accounts
- Nothing on your machine

FROM YOUR LAPTOP, CLOUD BACKEND

hydra run-test

Your checkout, your branch, real cloud machines. Hydra runs the test locally over public IPs, or on a runner you create in the cloud, which makes it the same as a Jenkins run.

Needs

- Docker or Podman, AWS access via Okta
- Cloud credentials for the backend
- Clean up when done, it costs money

FULLY LOCAL

docker or minicloud

Docker backend: cluster as containers, fastest for test logic. Minicloud: emulates EC2 and GCE APIs with KVM guests, so the real aws and gce code paths run.

Needs

- Docker, passwordless sudo, AWS access for S3
- Minicloud: KVM, about 22 GiB RAM, 80 GiB disk

Every way needs AWS access, even the local ones: SCT reads SSH keys and credentials from S3. Local results land in ~/sct-results/latest. Set SCT_ENABLE_ARGUS=false for local experiments.

PART 3 · SCT

Backends: one command, one flag

```
hydra run-test <module.Class.test> --backend <backend> --config test-cases/<case>.yaml
```

Backend	What it is	Notes
`aws · gce · azure · oci`	Public clouds, real VMs	aws most used, gce for artifacts and upgrades. From a laptop add the public-IP network config.
`xcloud`	ScyllaDB Cloud managed clusters	Provider and environment set by env vars
`k8s-eks · k8s-gke · k8s-local-kind`	Scylla Operator on Kubernetes	Managed clusters, or kind locally or on one VM
`baremetal`	Hosts that already exist, such as lab machines	Inventory from a JSON file or the keystore
`docker`	Containers on your machine	Not every nemesis works, see docs
`minicloud (aws or gce)`	Emulated cloud APIs, KVM guests	Same backend code as the real cloud, no instance cost

Per-backend settings and docs: README section 3 and docs/configuration_options.md.

PART 3 · SCT

What do I touch? Start at the top

Change a value, same test →

Env var in Argus, or edit the YAML

Duration, config path, versions, node counts. No code, no new job.

New variant of an existing test →

New test-case YAML

A new YAML under test-cases and a jenkinsfile pointing at it. Or clone the job in Argus and override values.

New behaviour or new result →

New or changed test class

Python, unit tests, a jenkinsfile, an Argus page. Follow docs/writing-a-test.md.

Checklist for a new test: test class, YAML with test_metadata, jenkinsfile, Argus page, unit tests, docs.

WRAP-UP

Three links to keep

Argus

argus.scylladb.com

Test pages, runs, results, Run button.

SCT README

github.com/scylladb/scylla-cluster-tests

Set up, first local run, everyday tasks, writing a test.

Getting started with SCT

Confluence · R&D space · Getting started with SCT

Internal setup: AWS access via Okta, first run, where to ask.